

```
import streamlit as st
import pandas as pd
from datetime import datetime
```

```
# إعدادات الصفحة لتكون متوافقة مع الأنظمة العربية (RTL)
st.set_page_config(
    page_title="نظام أودو المصغر | Odoo Lite",
    page_icon="👛",
    layout="wide",
    initial_sidebar_state="expanded"
)
```

```
# حقن كود CSS لجعل الواجهة بالكامل من اليمين إلى اليسار (RTL) وتغيير الألوان
لهوية أودو
```

```
st.markdown("""
<style>
body, div, p, h1, h2, h3, h4, th, td, label {
    direction: rtl;
    text-align: right;
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}
.stButton>button {
    background-color: #714B67; /* لون أودو الرسمي */
    color: white;
    border-radius: 4px;
    width: 100%;
}
.stButton>button:hover {
    background-color: #017E84; /* لون التفاعل في أودو */
    color: white;
}
.css-1d391kg {
```

```

text-align: right;
}
</style>
""", unsafe_allow_html=True)

# -----
# إدارة قاعدة البيانات المؤقتة (Session State)
# -----

if 'inventory' not in st.session_state:
    st.session_state.inventory = pd.DataFrame([
        {"كود المنتج": "PROD-001", "اسم المنتج": "ثلاجة توشيبا 16 قدم", "الفئة": "أجهزة منزلية", "الكمية الافتتاحية": 20, "سعر الشراء": 12000, "سعر البيع": 15000},
        {"كود المنتج": "PROD-002", "اسم المنتج": "شاشة إل جي 55 بوصة", "الفئة": "إلكترونيات", "الكمية الافتتاحية": 35, "سعر الشراء": 9500, "سعر البيع": 12000},
        {"كود المنتج": "PROD-003", "اسم المنتج": "خلاط مولينكس بفرامة", "الفئة": "أجهزة صغيرة", "الكمية الافتتاحية": 50, "سعر الشراء": 1100, "سعر البيع": 1500},
        {"كود المنتج": "PROD-004", "اسم المنتج": "تكييف شارب 2.25 حصان", "الفئة": "أجهزة منزلية", "الكمية الافتتاحية": 15, "سعر الشراء": 17500, "سعر البيع": 22000},
        {"كود المنتج": "PROD-005", "اسم المنتج": "مكنسة باناسونيك", "الفئة": "أجهزة صغيرة", "الكمية الافتتاحية": 4, "سعر الشراء": 3200, "سعر البيع": 4100}
    ])

if 'sales' not in st.session_state:
    st.session_state.sales = pd.DataFrame([
        {"رقم الفاتورة": "SO-001", "التاريخ والوقت": "14:22 11-07-2026", "العميل": "شركة الأمل للتجارة", "المنتج": "ثلاجة توشيبا 16 قدم", "سعر الوحدة": 15000, "الكمية": 2, "الخصم %": 0, "الإجمالي الصافي": 30000},
        {"رقم الفاتورة": "SO-002", "التاريخ والوقت": "15:10 11-07-2026", "العميل": "مؤسسة الهدى", "المنتج": "شاشة إل جي 55 بوصة", "سعر الوحدة": 12000, "الكمية": 5, "الخصم %": 5, "الإجمالي الصافي": 57000}
    ])

```

```

    ])

# -----
# احتساب المخزون الحالي ذكياً (الافتتاحي - المبيعات)
# -----

def get_current_inventory():
    df_inv = st.session_state.inventory.copy()
    df_sales = st.session_state.sales

    current_qty = []
    for idx, row in df_inv.iterrows():
        sold = df_sales[df_sales['المنتج'] == row['المنتج']]['الكمية'].sum()
        current_qty.append(row['الكمية الافتتاحية'] - sold)

    df_inv['الكمية الحالية'] = current_qty
    return df_inv

df_current_inv = get_current_inventory()

# -----
# القائمة الجانبية للتنقل بين تطبيقات أودو (Sidebar Navigation)
# -----

st.sidebar.title("👜 تطبيقات أودو")
app_mode = st.sidebar.radio("اختر التطبيق:", ["لوحة التحكم 📊", "حركات المبيعات 💰", "إدارة المخازن 📦", "Dashboard", "Inventory", "Sales"])

# -----
# 1. تطبيق لوحة التحكم (Dashboard)
# -----

if app_mode == "لوحة التحكم 📊 (Dashboard)":
    st.title("لوحة التحكم التنفيذية | Odoo Dashboard")

```

```

st.write("---")

# كروت الأداء الرئيسية
total_sales = st.session_state.sales['الإجمالي الصافي'].sum()
total_invoices = len(st.session_state.sales)
inventory_value = (df_current_inv['الكمية الحالية'] *
df_current_inv['سعر الشراء']).sum()

col1, col2, col3 = st.columns(3)
with col1:
    st.metric(label="💰 إجمالي المبيعات الحية",
value=f"{total_sales:,.2f} م.ج")
with col2:
    st.metric(label="📄 عدد أوامر البيع (Invoices)",
value=total_invoices)
with col3:
    st.metric(label="🏗️ قيمة المخزون الحالي (بسعر الشراء)",
value=f"{inventory_value:,.2f} م.ج")

st.write("---")

# تنبيهات المخزون الحرجة
st.subheader("🚨 تنبيهات النواقص (أقل من 5 قطع)")
low_stock = df_current_inv[df_current_inv['الكمية الحالية'] < 5]
if not low_stock.empty:
    st.dataframe(low_stock[['الكمية الحالية', 'اسم المنتج', 'كود المنتج']],
use_container_width=True)
else:
    st.success("حالة المخزن ممتازة! لا توجد نواقص حرجة")

# -----
# 2. تطبيق إدارة المخازن (Inventory)

```

```
# -----  
elif app_mode == "📦 إدارة المخازن (Inventory)":  
    st.title("📦 تطبيق إدارة المخازن والمنتجات")  
    st.write("---")
```

عرض المخزن الحالي

```
st.subheader("📋 جرد المخزن المباشر")  
st.dataframe(df_current_inv[['الكمية', 'الفئة', 'اسم المنتج', 'كود المنتج',  
    'الافتتاحية', 'الكمية الحالية', 'سعر الشراء', 'سعر البيع'],  
use_container_width=True)
```

إضافة منتج جديد

```
st.write("---")  
st.subheader("➕ إضافة منتج جديد للسیستم")  
with st.form("new_product_form"):  
    col1, col2 = st.columns(2)  
    with col1:  
        p_code = st.text_input("كود المنتج (SKU)")  
        p_name = st.text_input("اسم المنتج")  
        p_cat = st.selectbox("الفئة", ["أجهزة منزلية", "إلكترونيات", "أجهزة",  
            "صغيرة", "أخرى"])  
    with col2:  
        p_init = st.number_input("الكمية الافتتاحية", min_value=0,  
value=10)  
        p_cost = st.number_input("سعر الشراء", min_value=0.0,  
value=100.0)  
        p_price = st.number_input("سعر البيع الافتراضي", min_value=0.0,  
value=150.0)  
  
    submit_p = st.form_submit_button("حفظ المنتج بالدليل")  
    if submit_p:  
        new_p = {"الفئة": p_cat, "اسم المنتج": p_name, "كود المنتج": p_code,
```

```

p_cat, "الكمية الافتتاحية": p_init, "سعر الشراء": p_cost, "سعر البيع": p_price}
st.session_state.inventory =
pd.concat([st.session_state.inventory, pd.DataFrame([new_p])],
ignore_index=True)
st.success(f"بنجاح {p_name} تم تسجيل المنتج")
st.rerun()

```

```

# -----

```

3. تطبيق حركات المبيعات (Sales)

```

# -----

```

```

elif app_mode == "💰 حركات المبيعات (Sales)":
st.title("💰 تطبيق المبيعات ونقاط البيع")
st.write("---")

```

عرض الفواتير المسجلة

```

st.subheader("📋 سجل أوامر البيع المسجلة")
st.dataframe(st.session_state.sales, use_container_width=True)

```

إنشاء أمر بيع جديد (Sales Order)

```

st.write("---")
st.subheader("🛒 إنشاء فاتورة مبيعات جديدة")

```

```

with st.form("new_sale_form"):
col1, col2 = st.columns(2)
with col1:
so_number = f"SO-{len(st.session_state.sales) + 1:03d}"
st.info(f"رقم الفاتورة التلقائي {so_number}")
customer = st.text_input("اسم العميل", value="عميل نقدي")

```

قائمة منسدلة ذكية بالمنتجات المتاحة من المخزن

```

product_options = df_current_inv['المنتج'].tolist()
selected_product = st.selectbox("اختر المنتج (Dropdown)",

```

product_options)

with col2:

```
# جلب سعر المنتج المختار تلقائياً
default_price = float(df_current_inv[df_current_inv['اسم المنتج'] == selected_product]['سعر البيع'].values[0])
unit_price = st.number_input("سعر الوحدة (تلقائي ويمكن تعديله)",
value=default_price)

qty = st.number_input("الكمية المطلوبة", min_value=1, value=1)
discount = st.number_input("(% الخصم)", min_value=0,
max_value=100, value=0)

submit_s = st.form_submit_button("تأكيد أمر البيع وترحيله للمخزن")
if submit_s:
    # التحقق من توفر كمية في المخزن قبل البيع
    available_qty = df_current_inv[df_current_inv['اسم المنتج'] ==
selected_product]['الكمية الحالية'].values[0]
    if qty > available_qty:
        st.error(f"خطأ: الكمية المطلوبة أكبر من المتاح في المخزن! المتاح حالياً {available_qty} قطع فقط")
    else:
        total_net = (unit_price * qty) * (1 - discount/100)
        now_str = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        new_sale = {
            "رقم الفاتورة": so_number, "التاريخ والوقت": now_str, "العميل": customer,
            "المنتج": selected_product, "سعر الوحدة": unit_price, "الكمية": qty,
            "الخصم": discount, "الإجمالي الصافي": total_net
        }
```

```
}
```

```
    st.session_state.sales =  
pd.concat([st.session_state.sales, pd.DataFrame([new_sale]),  
ignore_index=True)  
    st.success(f"تم تأكيد الفاتورة {so_number} وخصم {qty} من قطع  
المخزن")  
    st.rerun()
```